

TCP Client Experiment

TCP Client Experiment

- 1. TCP Client Overview
- 2. Core Concepts
 - 2.1 TCP Connection Lifecycle
 - 2.2 The "Packet Sticking" Problem of Stream Protocols
 - 2.3 `write()` vs `print()`
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Analysis
 - 5.1 Configuration and Global Variables
 - 5.2 `setup()` — Establish TCP Connection
 - 5.3 `loop()` — Send and Receive
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Operation Steps
 - 6.3 Verification Methods
- 7. Common Issues

1. TCP Client Overview

TCP (Transmission Control Protocol) is a **connection-oriented, reliable, byte-stream** transport layer protocol. It establishes connections through three-way handshake, ensuring data arrives in order and intact.

TCP is the foundation of IoT communication. Whether it's HTTP web requests, MQTT message publishing, OTA firmware upgrades, or database remote connections, they all rely on TCP's reliable transmission at the underlying layer. In industrial automation, smart home, and remote monitoring scenarios, devices report sensor data and receive control commands, requiring "no data loss, no order disruption" - this is TCP's core value. UDP is fast but unreliable; TCP's three-way handshake overhead is worth it for the **end-to-end acknowledgment and retransmission mechanism**, which is irreplaceable in most IoT scenarios.

In this experiment, ESP32-S3 acts as a TCP client to actively connect to the server, sending one data message per second and receiving server responses. The `WiFiClient` class encapsulates lwIP's TCP socket, with connection, sending, receiving, and disconnection detection all completed through simple method calls.

Typical Application Scenarios:

Application Type	Example
HTTP Requests	Browser accessing web pages (HTTP is essentially TCP at the underlying layer)
Remote Control	SSH, Telnet remote terminals
Data Collection	Reliable sensor data upload
File Transfer	FTP, custom binary protocols

2. Core Concepts

2.1 TCP Connection Lifecycle

A TCP client goes through 5 phases from connection to closure, each corresponding to a `WiFiClient` method:

connect(ip, port)

establish connection

↓

underlying: three-way handshake SYN → SYN-ACK → ACK, blocking wait, returns true on success

write / print

send data

↓

full-duplex: both parties can send/receive simultaneously, write sends raw bytes, print sends formatted text

available() / read()

receive data

↓

available() returns number of readable bytes, non-blocking, returns 0 if no data

connected()

liveness check

↓

returns false if remote endpoint has closed (received FIN/RST) or network timeout

stop()

close connection

underlying: four-way handshake FIN → ACK ← FIN → ACK, release socket resources

Method	Function	Blocking?
<code>connect(ip, port)</code>	Initiate three-way handshake, establish TCP connection	Yes
<code>write(buf, len)</code>	Send raw bytes	No (writes to send buffer)
<code>print(str)</code>	Send formatted text	No
<code>available()</code>	Query number of readable bytes in receive buffer	No
<code>read(buf, len)</code>	Read data from receive buffer	No
<code>connected()</code>	Check if connection is alive	No
<code>stop()</code>	Actively close connection (four-way handshake)	No

2.2 The "Packet Sticking" Problem of Stream Protocols

TCP is a **stream** (Stream) protocol, unlike UDP which preserves message boundaries. If the sender does two `write("Hello")` and `write("world")`, the receiver may receive `"Helloworld"` in one `read()` (packet sticking), or may receive three fragments `"Hel"`, `"lowor"`, `"ld"` (fragmentation). **The application layer must define message boundaries itself** (such as newline characters, length prefixes, etc.).

2.3 `write()` vs `print()`

Function	Purpose	Applicable Scenario
<code>write(buf, len)</code>	Send raw bytes	Binary protocols
<code>print(data)</code>	Formatted text (numbers → strings)	HTTP, text protocols

Function	Purpose	Applicable Scenario
<code>println(data)</code>	<code>print</code> + <code>\r\n</code> at end	HTTP line endings

3. Hardware Dependencies

Pure software experiment. Need to connect to router via WIFI-STA first, client and server must be on the same local area network.

4. Project Architecture and Flow

```
setup()
├─ connectWiFi()
└─ client.connect(SERVER_IP, SERVER_PORT)  // three-way handshake

loop()
├─ if disconnected → stop() → mark for reconnect
├─ client.write(send_buf)                  // send raw bytes
├─ if client.available() → client.read()   // non-blocking read
└─ delay(1000)
```

5. Source Code Analysis

Complete source code: `Source code → Arduino → 3.Network Communication → TCP_Client → TCP_Client.ino`

5.1 Configuration and Global Variables

```
#include <WiFi.h>
WiFiClient client;
const char* SERVER_IP = "192.168.11.183";
uint16_t    SERVER_PORT = 8080;
```

5.2 `setup()` — Establish TCP Connection

`client.connect()` initiates three-way handshake, returns `true` on success, returns `false` on failure (server not listening/firewall rejection).

```

void setup() {
  Serial.begin(115200);
  connectWiFi();

  if (!client.connect(SERVER_IP, SERVER_PORT)) {
    Serial.println("Failed to connect to the server!");
    return;
  }
  Serial.println("Server Connected Successfully!");
}

```

5.3 loop() — Send and Receive

```

void loop() {
  if (!client.connected()) {          // detect disconnection
    Serial.println("TCP Server disconnected!");
    client.stop();
    delay(2000);
    ESP.restart();
    return;
  }

  client.write("ESP32 TCP Client Data"); // send raw bytes

  if (client.available()) {           // non-blocking read
    char buf[128];
    int len = client.read(buf, sizeof(buf) - 1);
    buf[len] = '\0';
    Serial.printf("Receive: %s\n", buf);
  }
  delay(1000);
}

```

`client.connected()` returns `false` when: server FIN received, RST received, or multiple retransmissions with no ACK timeout. `ESP.restart()` is used for overall reset and reconnect after disconnection—this is the cleanest approach.

6. Operation Flow

6.1 Build and Flash

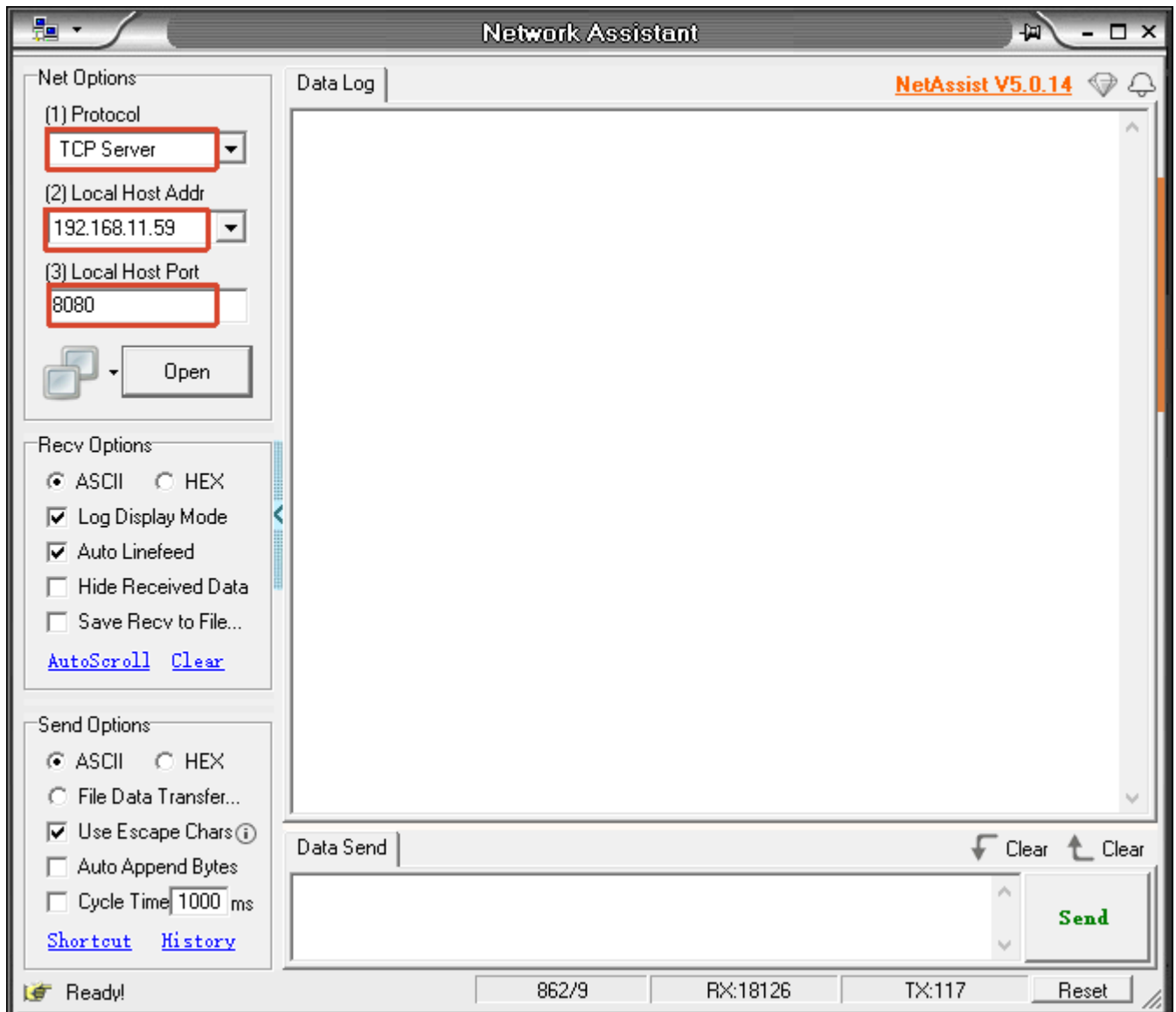
Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

1. Modify server IP to actual address
2. First use network debugging assistant or TCP server program to listen on port 8080
3. Build and flash

6.2 Operation Steps

Step 1: Open network debugging assistant , configure TCP server

Open network debugging assistant on PC, configure with following parameters:



Parameter	Value
Protocol Type	TCP Server
Local IP Address	PC's LAN IP (e.g., 192.168.11.183)
Local Port	8080

Click "Connect" button, network debugging assistant starts listening on TCP port 8080.

Step 2: Modify server IP in code, flash program

Open Arduino IDE, modify `SERVER_IP` in code to PC's actual LAN IP address, then compile and flash to ESP32-S3 development board.

```
// 配置区 | Config Area
const char* WIFI_SSID = "Yahboom2";           // WiFi SSID
const char* WIFI_PASS = "yahboom890729";      // WiFi 密码 | WiFi password
const char* SERVER_IP = "192.168.11.59";      // 目标 TCP 服务器 IPv4 地址 | Target TCP server IPv4 address
uint16_t SERVER_PORT = 8080;                  // 目标 TCP 端口 | Target TCP port
```

Step 3: Open serial monitor, observe connection and send/receive

After flashing, open serial monitor (baud rate 115200), ESP32 will automatically connect to TCP server, after successful connection sends one data message per second and receives replies:

```
11:30:16.823 -> Connecting to WiFi: Yahboom2 (Attempt 1)
11:30:17.877 -> ...
11:30:18.885 -> WiFi Connected successfully!
11:30:18.885 -> Device IP: 192.168.11.229
11:30:18.885 -> Target Server: 192.168.11.183:8080
11:30:18.930 -> Server Connected Successfully!
11:30:18.930 -> Send data: ESP32 TCP Client Data
11:30:19.942 -> Send data: ESP32 TCP Client Data
11:30:20.950 -> Send data: ESP32 TCP Client Data
...
```

```
Connecting to WiFi: Yahboom2 (Attempt 1)
...
WiFi Connected successfully!
Device IP: 192.168.11.181
Target Server: 192.168.11.59:8080
Server Connected Successfully!
Send data: ESP32 TCP Client Data
Send data: ESP32 TCP Client Data
Send data: ESP32 TCP Client Data
Send data: ESP32 TCP Client Data
Send data: ESP32 TCP Client Data
Send data: ESP32 TCP Client Data
```

At this point, enter any content in the send area of network debugging assistant and send, serial monitor will print data received by ESP32.

```
[2026-08-06 19:27:53.289]# Client 192.168.11.181 :49723 online.  
[2026-08-06 19:27:53.291]# RECV ASCII/21 from 192.168.11.181 :49723 <<<  
ESP32 TCP Client Data  
[2026-08-06 19:27:54.293]# RECV ASCII/21 from 192.168.11.181 :49723 <<<  
ESP32 TCP Client Data  
[2026-08-06 19:27:55.301]# RECV ASCII/21 from 192.168.11.181 :49723 <<<  
ESP32 TCP Client Data  
[2026-08-06 19:27:56.307]# RECV ASCII/21 from 192.168.11.181 :49723 <<<  
ESP32 TCP Client Data  
[2026-08-06 19:27:57.304]# RECV ASCII/21 from 192.168.11.181 :49723 <<<  
ESP32 TCP Client Data  
[2026-08-06 19:27:58.313]# RECV ASCII/21 from 192.168.11.181 :49723 <<<  
ESP32 TCP Client Data  
[2026-08-06 19:27:59.301]# RECV ASCII/21 from 192.168.11.181 :49723 <<<  
ESP32 TCP Client Data  
[2026-08-06 19:28:00.294]# RECV ASCII/21 from 192.168.11.181 :49723 <<<  
ESP32 TCP Client Data
```

Data interaction:

```
Send data: ESP32 TCP Client Data  
Receive server data: Hello YAHBOOM  
Send data: ESP32 TCP Client Data  
Send data: ESP32 TCP Client Data  
Receive server data: Hello YAHBOOM  
Send data: ESP32 TCP Client Data  
Send data: ESP32 TCP Client Data
```

Step 4: Test disconnection detection

Close TCP server of network debugging assistant (click "Disconnect"). After ESP32 detects disconnection, it prints `TCP Server disconnected!`, automatically restarts after 2 seconds and attempts to reconnect.

```
TCP Server disconnected!
```

6.3 Verification Methods

Operation	Expected Result
Power on when server is online	Connection successful, send/receive once per second
Disconnect server	Print "disconnected", <code>ESP.restart()</code> resets and reconnects

7. Common Issues

Symptom	Cause	Solution
<code>connect()</code> never returns	Firewall drops SYN packet with no response	Check server firewall settings
Received data stuck together	Inherent characteristic of TCP stream protocol	Add message boundary at application layer (e.g., <code>\n</code> separator)
<code>connected()</code> returns true but write fails	Connection closed but not detected (half-open connection)	Enable TCP keepalive or periodic heartbeat